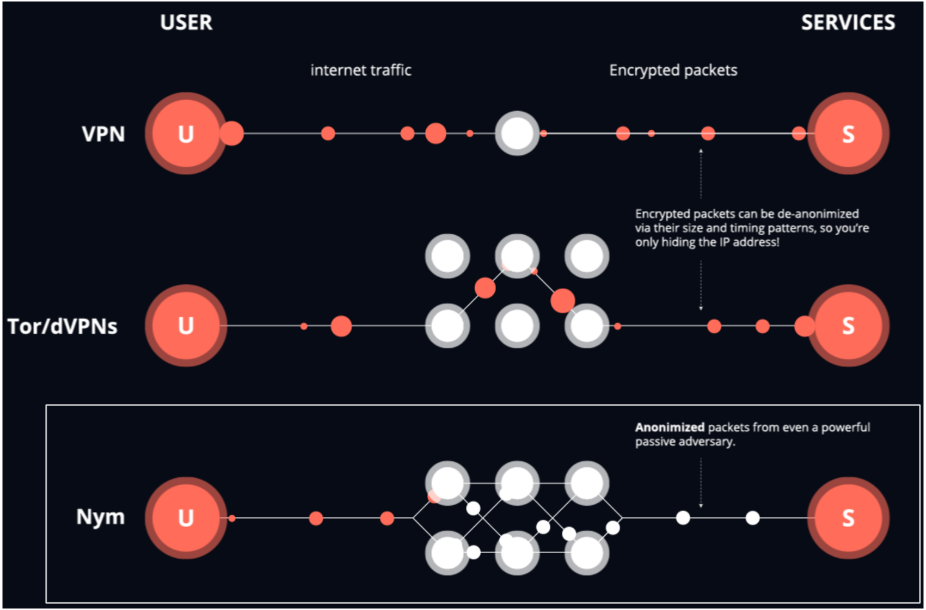
Sphinx Packets

Decentralized Header Construction

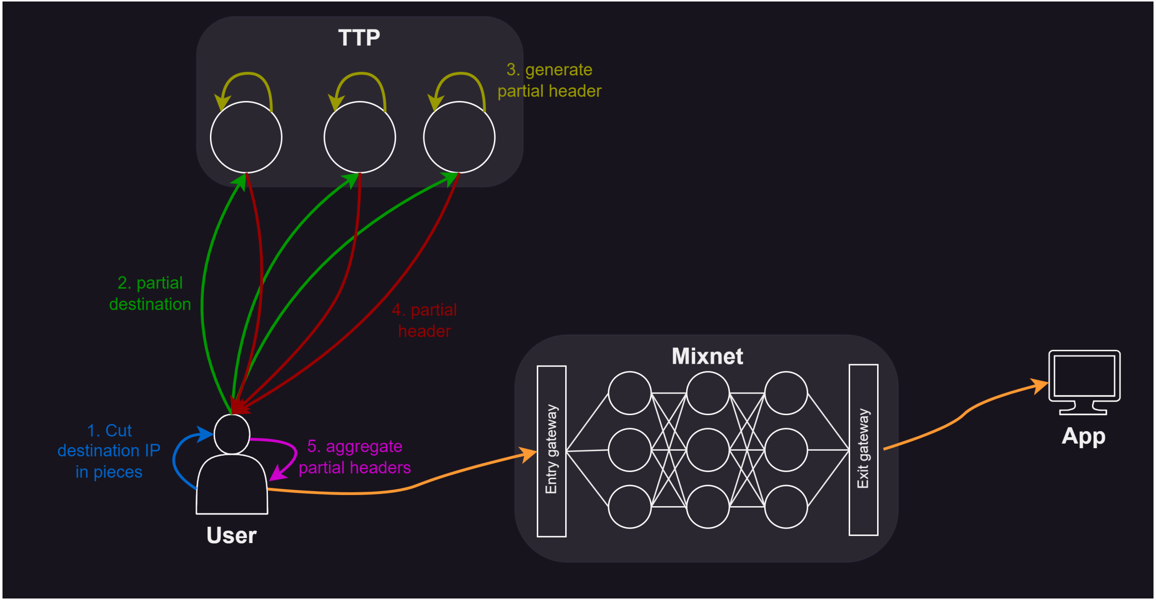
Aurélien Chassagne

February 18, 2025

Mixnet



USER
SERVICES
internet traffic
Encrypted packets
VPN
U
S
Encrypted packets can be de-anonimized via their size and timing patterns, so you're only hiding the IP address!
Tor/dVPNs
U
S
Anonimized packets from even a powerful passive adversary.
Nym
U
S


TTP
3. generate
partial header
2. partial
destination
4. partial
header
Mixnet
Entry gateway
Exit gateway
App
1. Cut
destination IP
in pieces
5. aggregate
partial headers
User

Desired properties

- Generic properties
  - **Correctness**: schema works without adversary
  - **Compactness**: Minimal overhead
  - **Efficiency**: Easy and fast to compute (e.g. XOR, hash, exponentiation,...)

Desired properties

- Generic properties
  - **Correctness**: schema works without adversary
  - **Compactness**: Minimal overhead
  - **Efficiency**: Easy and fast to compute (e.g. XOR, hash, exponentiation,...)
- Depends on the mixnode
  - **Forward / reply Undistinguishibility**: Cannot distinguish forward from reply packet
  - **Replay attack resistant:**: Cannot reused previous packet

Desired properties

- Generic properties
  - **Correctness**: schema works without adversary
  - **Compactness**: Minimal overhead
  - **Efficiency**: Easy and fast to compute (e.g. XOR, hash, exponentiation,...)
- Depends on the mixnode
  - **Forward / reply Undistinguishibility**: Cannot distinguish forward from reply packet
  - **Replay attack resistant:**: Cannot reused previous packet
- Depends on the header
  - **Integrity**: Maximum size path
  - **Wrap-resistance**: Unable to increase the intial path
  - **Unlinkability**: Cannot link incoming and outgoing packet from a mixnode

Unlinkability compromised

Original schema

Images/res/origin_gamma2.png

My schema

Images/res/test_gamma2.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

Images/res/test_ip.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

Images/res/test_gamma3.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

Images/res/test_n3.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

Images/res/test_gamma2.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

Images/res/test_n2.png

Unlinkability compromised

n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

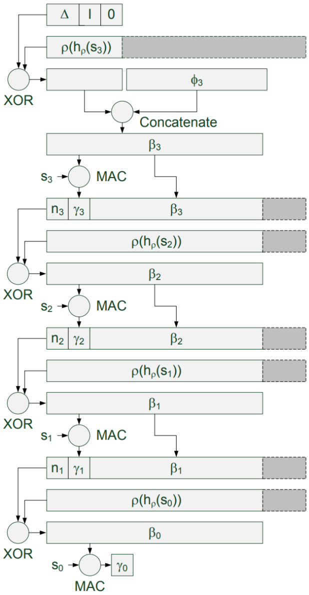
Images/res/test_gamma1.png

Unlinkability compromised

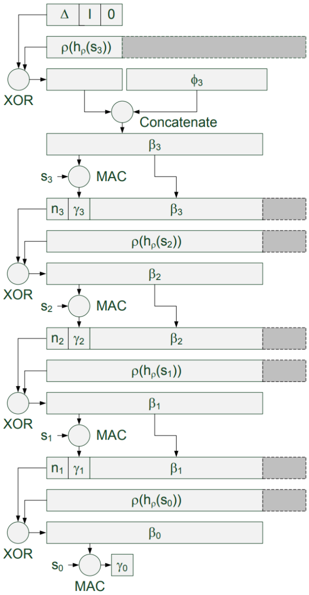
n_1	γ_1	n_2	γ_2	n_3	γ_3	IP

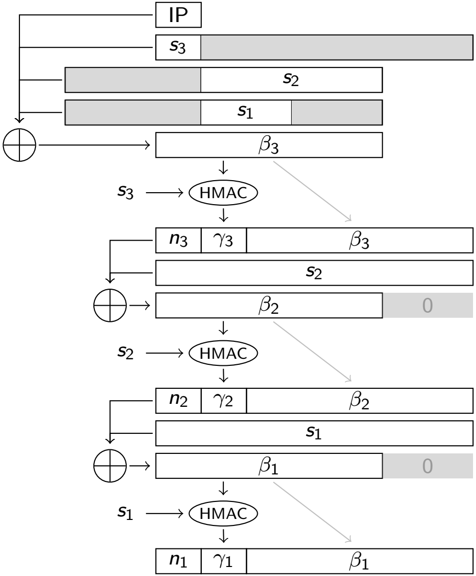
Images/res/test_n1.png

Reproduce article's figure

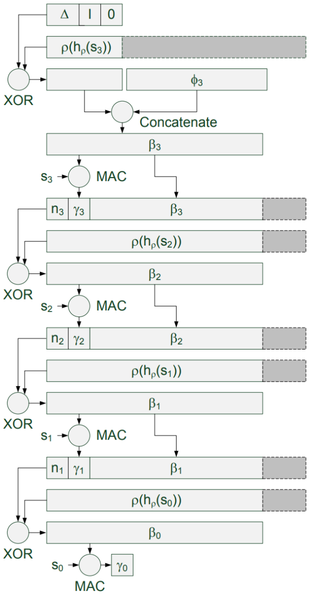


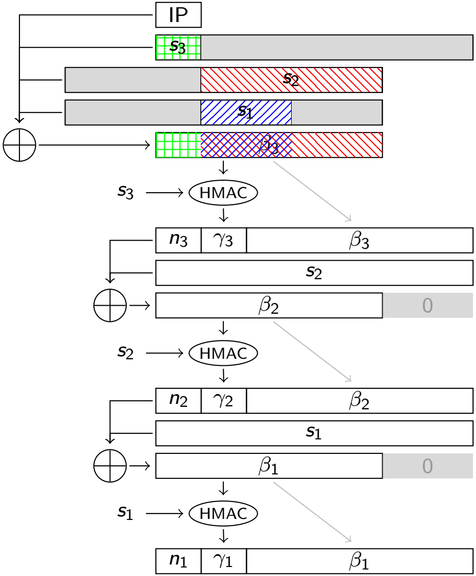
Reproduce article's figure



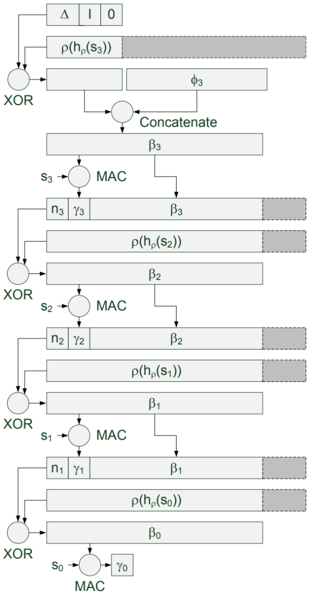


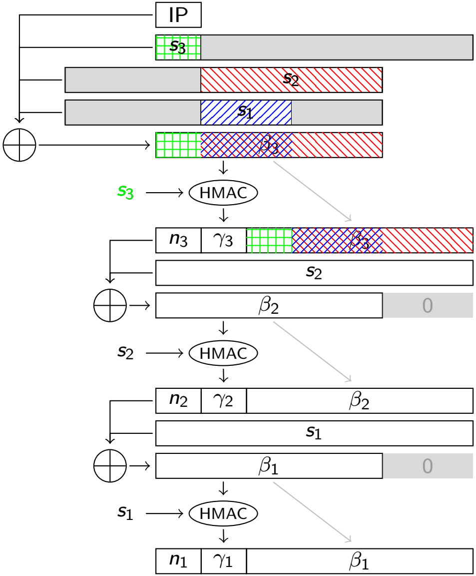
Reproduce article's figure



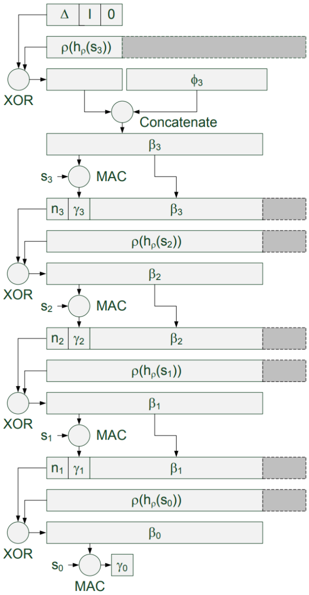


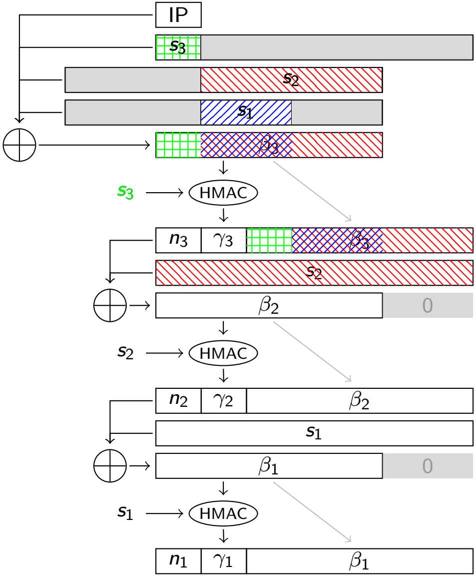
Reproduce article's figure



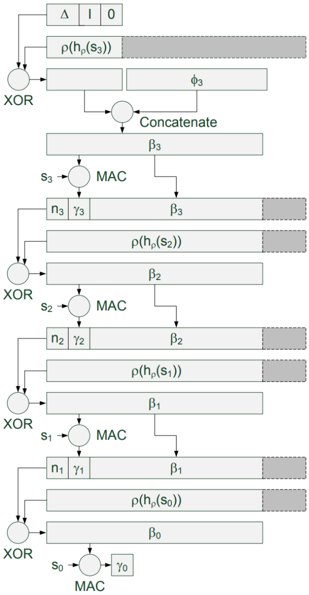


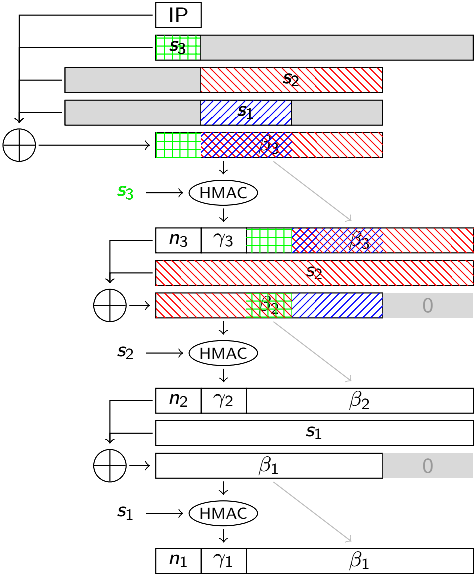
Reproduce article's figure



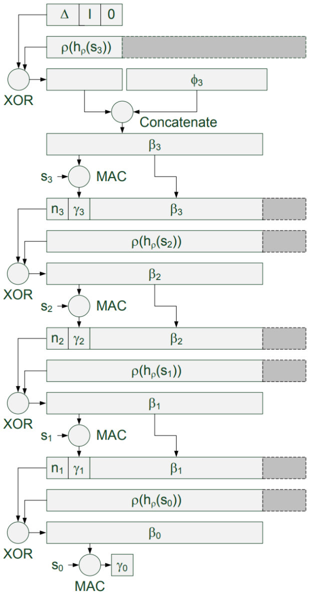


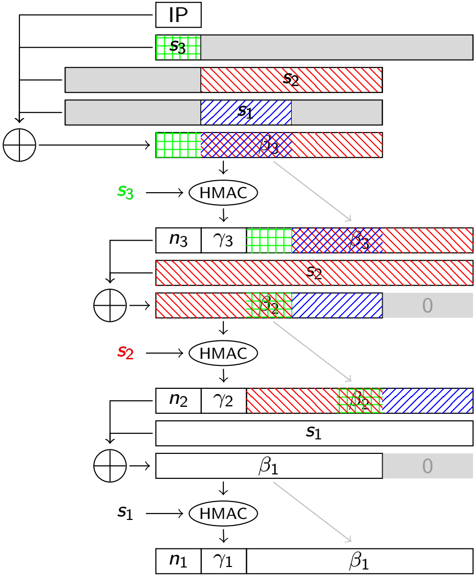
Reproduce article's figure



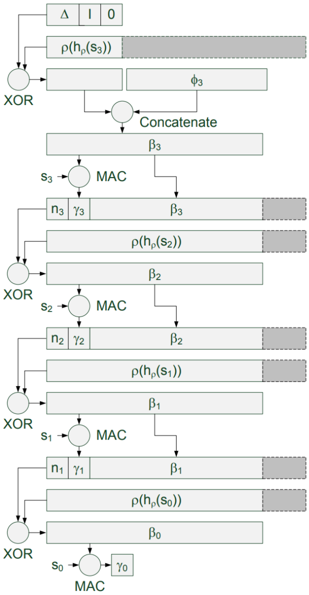


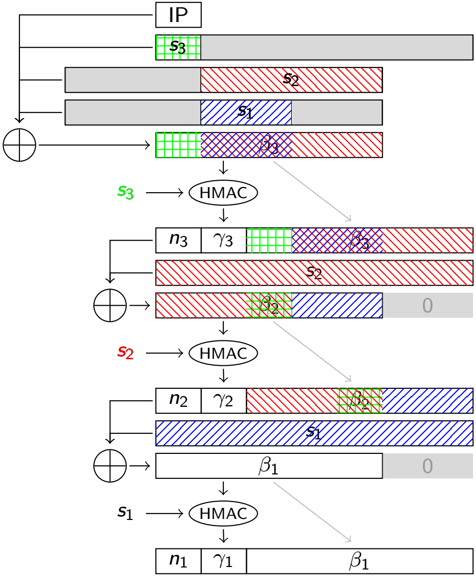
Reproduce article's figure



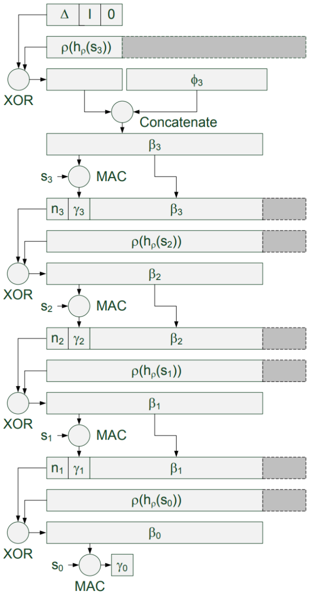


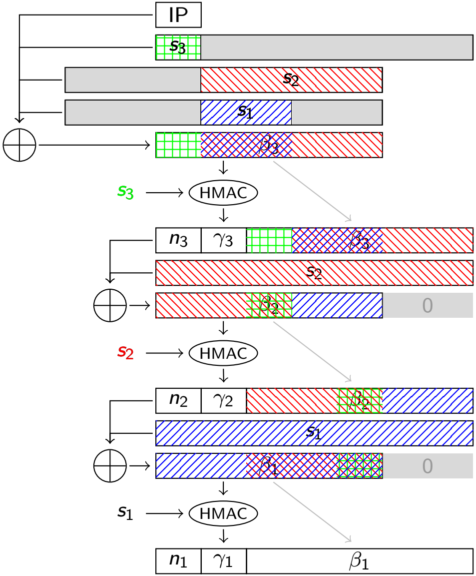
Reproduce article's figure



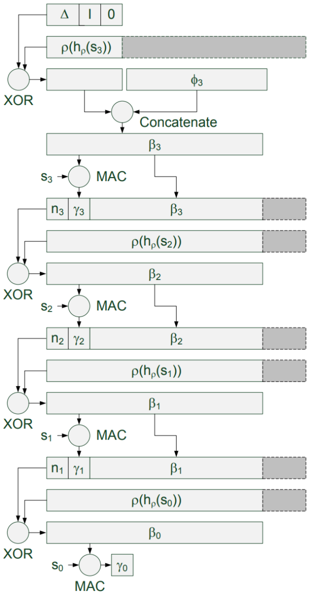


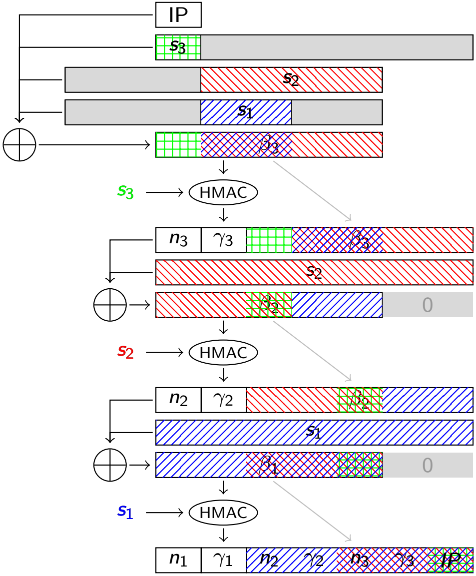
Reproduce article's figure



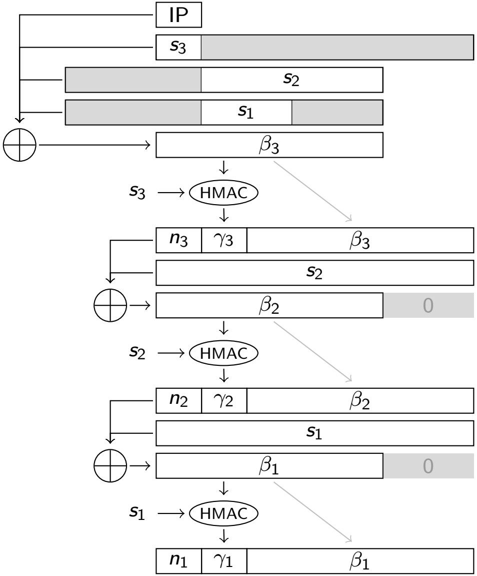


Reproduce article's figure



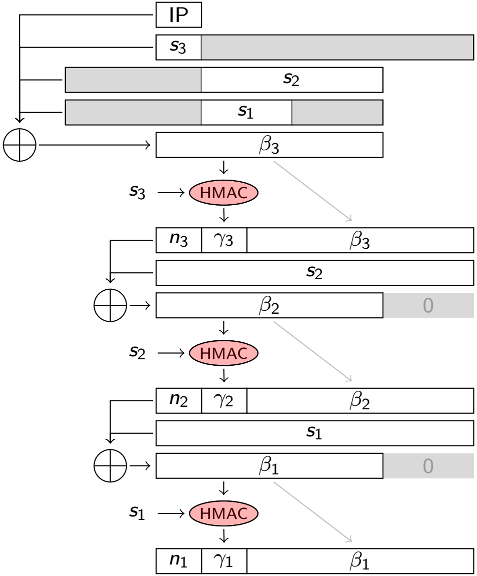


Adapting HMAC



Adapting HMAC

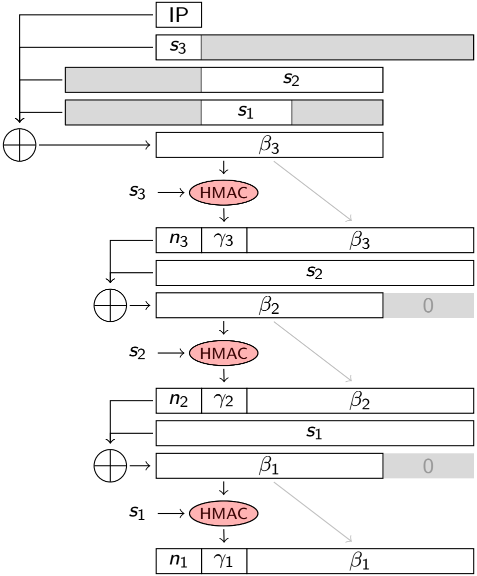
Main problem: Decentralizing a **Hash** ?



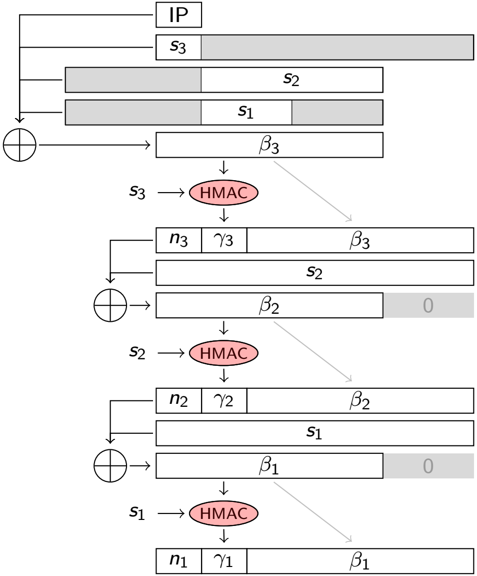
Adapting HMAC

Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties**
  to split computation and aggregate results.



Adapting HMAC



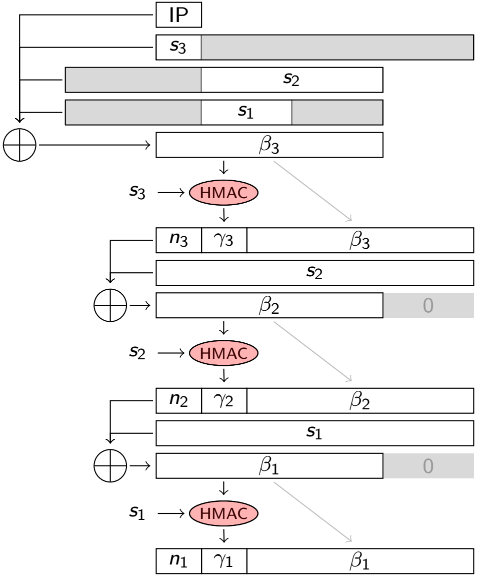
Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).

Adapting HMAC

Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties**
  to split computation and aggregate results.
- Secure homomorphic hash **seems impractical**
  (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

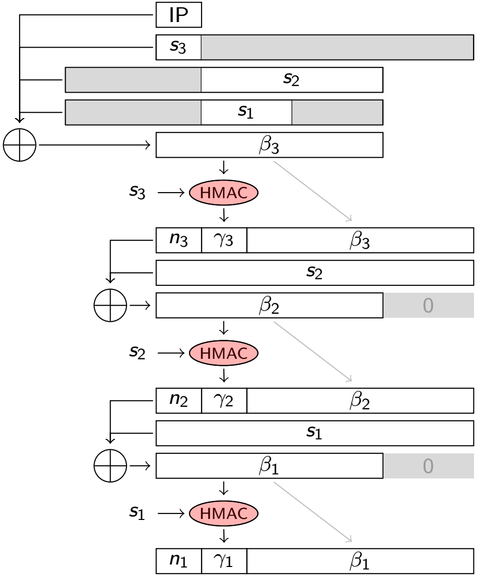


Adapting HMAC

Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

1. **RSA**
2. **ElGamal**
3. **Paillier**



Adapting HMAC

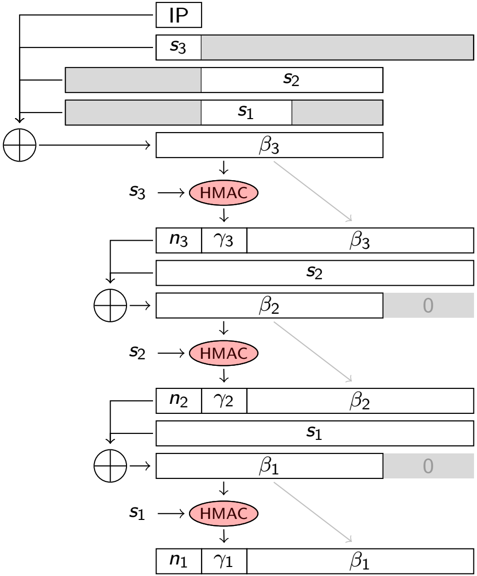
Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

1. **RSA**
2. **ElGamal**
3. **Paillier**

$$\begin{aligned}\mathcal{E}(m_1)\cdot\mathcal{E}(m_2) &= (g^{m_1} r_1^n)(g^{m_2} r_2^n) \bmod n^2 \\ &= g^{m_1+m_2}(r_1 r_2)^n \bmod n^2 \\ &= \mathcal{E}(m_1+m_2).\end{aligned}$$

Problem: Mix of different operations... order matters!



Adapting HMAC

Main problem: Decentralizing a **Hash** ?

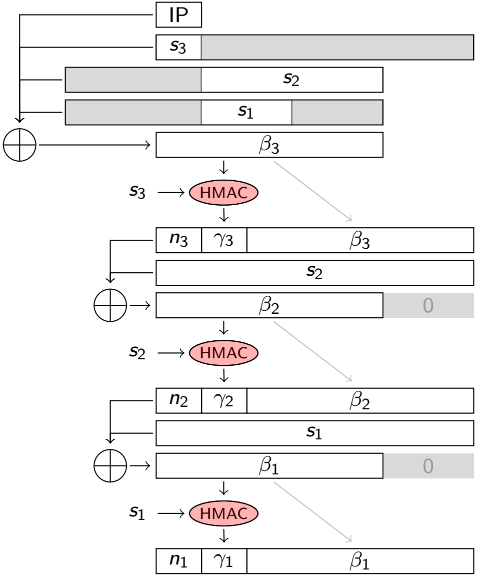
- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

1. **RSA**
2. **ElGamal**

$$\mathcal{E}(m_1) \cdot \mathcal{E}(m_2) = (g^{r_1}, m_1 \cdot h^{r_1})(g^{r_2}, m_2 \cdot h^{r_2})$$
$$= (g^{r_1+r_2}, (m_1 \cdot m_2)h^{r_1+r_2})$$
$$= \mathcal{E}(m_1 \cdot m_2)$$

3. **Paillier**

Limitation: Increase ciphertext size...



Adapting HMAC

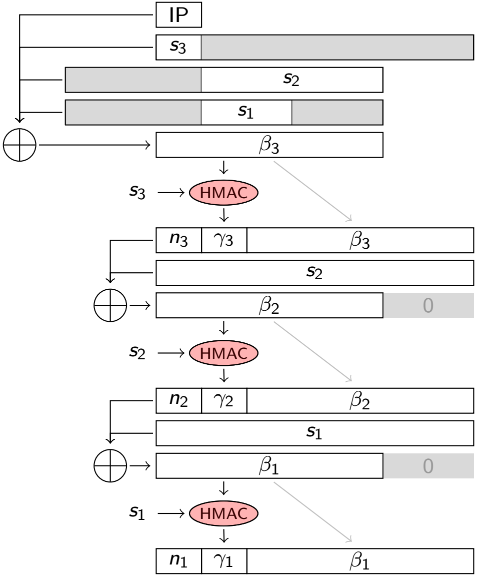
Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

1. **RSA**

$$\begin{aligned}\mathcal{E}(m_1)\cdot\mathcal{E}(m_2) &= m_1^e m_2^e \bmod n\\ &= (m_1 m_2)^e \bmod n\\ &= \mathcal{E}(m_1\cdot m_2)\end{aligned}$$

2. **ElGamal**
3. **Paillier**



Adapting HMAC

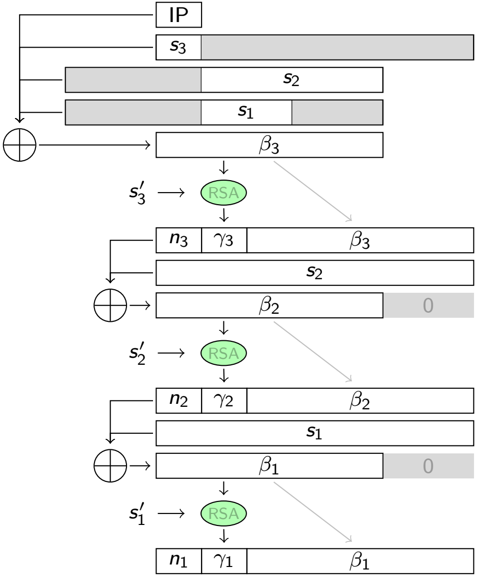
Main problem: Decentralizing a **Hash** ?

- Need **homomorphic properties** to split computation and aggregate results.
- Secure homomorphic hash **seems impractical** (no promising solutions found).
- Exploring **homomorphic encryption** as an alternative.

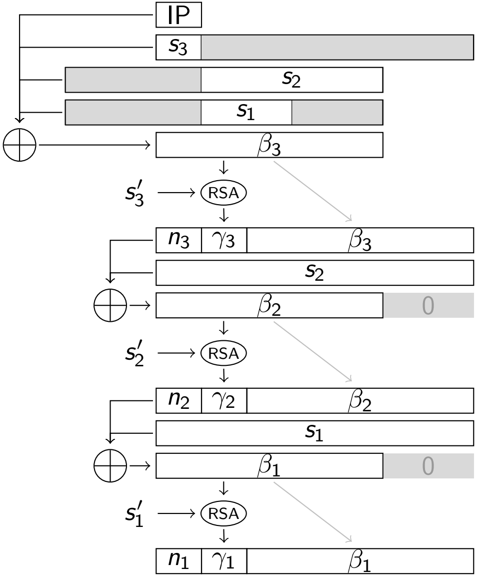
  1. **RSA**
  2. **ElGamal**
  3. **Paillier**

Selected solution: RSA for integrity tag

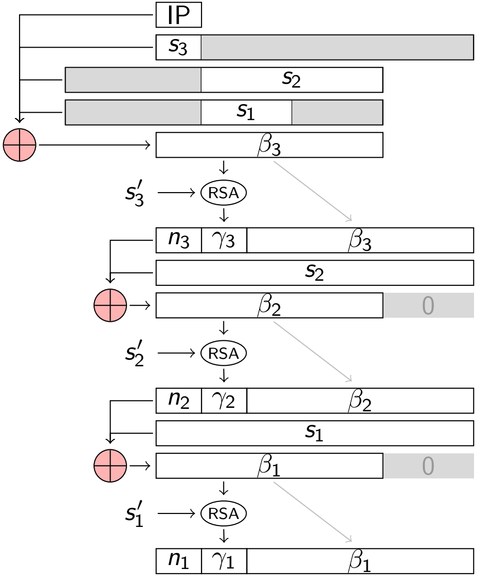
NB: s_i is different for each TTP but RSA required the same e...
Thus, create a new shared secret s_i' common to all TTP



Adapting XOR

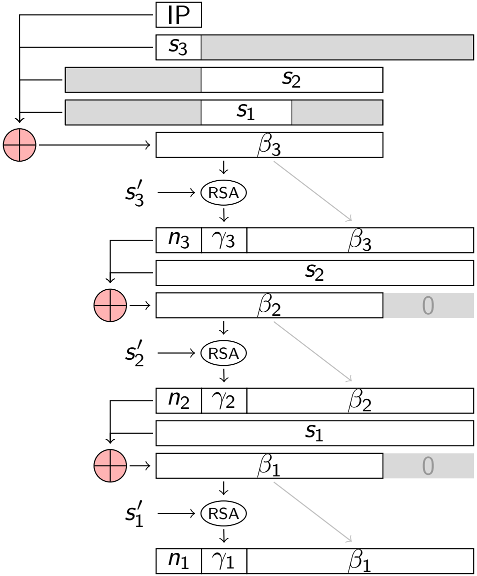


Adapting XOR



Adapting XOR

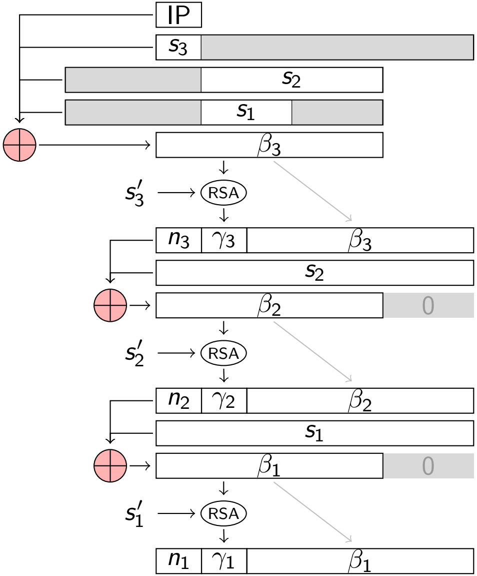
Since we use **RSA** for integrity tag γ_i



Adapting XOR

Since we use **RSA** for integrity tag γ_i

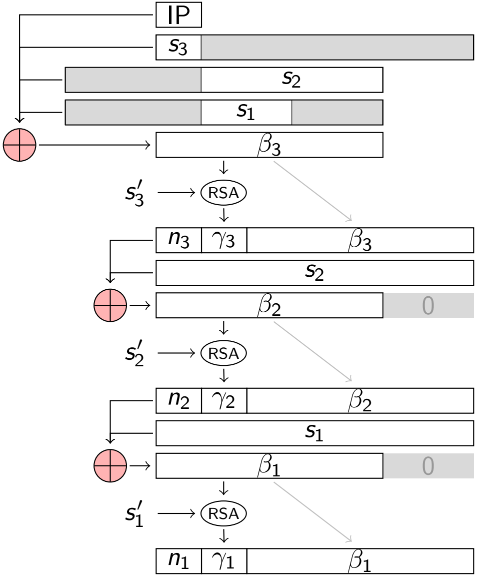
$$\begin{aligned}\mathcal{E}(m_1)\cdot\mathcal{E}(m_2) &= m_1^e m_2^e \bmod n\\ &= (m_1 m_2)^e \bmod n\\ &= \mathcal{E}(m_1\cdot m_2)\end{aligned}$$



Since we use **RSA** for integrity tag γ_i

$$\begin{aligned}\mathcal{E}(m_1)\cdot\mathcal{E}(m_2) &= m_1^e m_2^e \bmod n\\ &= (m_1 m_2)^e \bmod n\\ &= \mathcal{E}(m_1\cdot m_2)\end{aligned}$$

Modular multiplication of integrity tags gives integrity tag of headers' modular product.



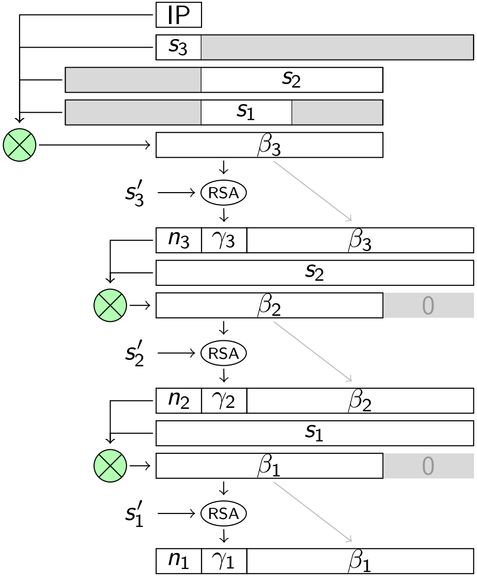
Adapting XOR

Since we use **RSA** for integrity tag γ_i

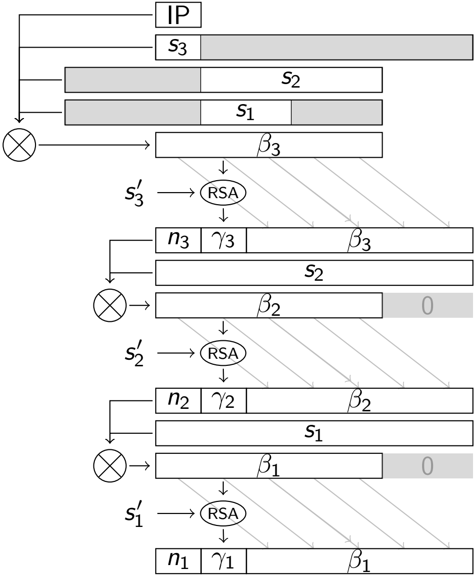
$$\begin{aligned} \mathcal{E}(m_1)\cdot\mathcal{E}(m_2) &= m_1^e m_2^e \bmod n \\ &= (m_1 m_2)^e \bmod n \\ &= \mathcal{E}(m_1\cdot m_2) \end{aligned}$$

Modular multiplication of integrity tags gives integrity tag of headers' modular product.

Thus, header elements must be combined via modular multiplication rather than XOR.

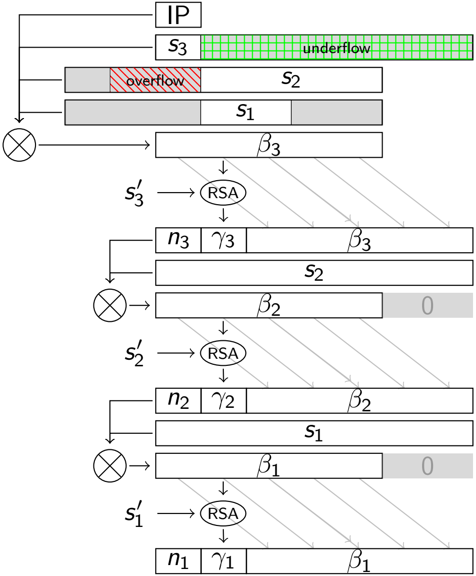


Handle overflows

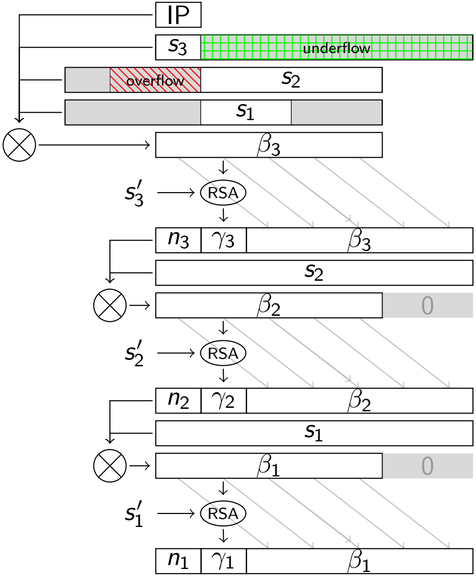


Handle overflows

- **Current Challenge:** **Overflow** issues



- **Current Challenge: Overflow** issues
- Handling these issues is challenging. It may lead to **information loss** (further research is required).



Handle overflows

- **Current Challenge:** **Overflow** issues
- Handling these issues is challenging. It may lead to **information loss** (further research is required).
- **Proposed solution:** Simplify by **dividing data into small chunks** and processing each chunk modulo its size.